Optimal Parallel Algorithms for the 3D Euclidean Distance Transform on the CRCW and EREW PRAM Models *

Yuh-Rau Wang†‡, Shi-Jinn Horng†§, Yu-Hua Lee†, and Pei-Zong Lee§

†Department of Electrical Engineering, National Taiwan University of Science and Technology, Taipei, Taiwan.
‡Department of Electronic Engineering, Lan Yang Institute of Technology, I-Lan, Taiwan.
§Institute of Information Science, Academia Sinica, Taipei, Taiwan.

Abstract

In this paper, based on the dimensionality reduction technique and the solution for proximate points problem, we achieve the optimality of the three-dimensional Euclidean distance transform (3D_EDT) *computation. For an $N \times N \times N$ binary image array, our parallel algorithms for both 3D_EDT and its applications such as building up Voronoi diagram and Voronoi polyhedra, finding all maximal empty spheres and the largest empty sphere, and computing the distance-based medial axis transform* (MAT) *in a 3-D binary image, all of them can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N^3}{logN}$ EREW processors. To the best of our knowledge, all results described above are the best that never found before. As for the n-dimensional Euclidean distance transform* (nD_EDT) *and its applications as described above of a binary image of size N^n, all of them can be computed in $O(nloglogN)$ time using $\frac{N^n}{loglogN}$ CRCW processors or in $O(nlogN)$ time using $\frac{N^n}{logN}$ EREW processors.*

Keywords: Euclidean Distance Transform, Parallel Algorithm, Voronoi Diagram, Proximate point, EREW PRAM, CRCW PRAM.

*The correspondence address is : Prof. Shi-Jinn Horng, Department of Electrical Engineering, National Taiwan University of Science and Technology, 43, Section 4, Kee-Lung Road, Taipei, Taiwan, R. O. C. Tel: (02)2737-6700, FAX: (02)2737-6699, E-mail: horng@mouse.ee.ntust.edu.tw or yrwang@mail.fit.edu.tw. The work was partly supported by National Science Council under the contract no. NSC-89-2213-E011-007. Part of this work was carried out while the second author was visiting the Institute of Information Science, Academia Sinica, Taipei, Taiwan, during July 1, 2001 to September 30, 2001.

1 Introduction

Given a computational problem P, let a sequential time complexity of P be $T^*(n)$. A sequential algorithm is termed *time-optimal* [10] if its time bound $O(T^*(n))$ cannot be improved. The main parallel complexity measure for evaluating the performance of an algorithm is the amount $W(n)$ of work performed by this algorithm, denoted by the product $p \times T_p(n)$, where $T_p(n)$ is the time complexity of a parallel algorithm using p processors. A parallel algorithm is termed *optimal* or *work-optimal* [10] if the work $W(n)$ required by this algorithm satisfies $W(n) = \Theta(T^*(n))$, where $T^*(n)$ is the running time of the fastest sequential algorithm for the problem.

Processing digital image data from three-dimensional (*3-D*, for short) objects is an important task in the image processing and the computer vision fields. The distance transform (*DT*, for short) first introduced by *Rosenfeld* and *Pfaltz* [15] is one of the most important elementary operations. The *Euclidean distance transform* (*EDT*, for short) is based on the Euclidean metrics and is an operation that converts an image consisting of black and white pixels (or voxels) to an image where each pixel (voxel) has a value that represents the Euclidean distance to its nearest 1-pixel (or 1-voxel). It is prohibitively time consuming on the 3D_EDT computation.

Recently, many papers were introduced for constructing the 2D_EDT of a 2-D binary image of size $N \times N$. Hirata [9], and Breu et al. [5] presented $O(N^2)$ time sequential algorithms for computing the EDT. On the EREW PRAM model, Lee et al. [13] presented an $O(log^2 N)$ time algorithm using n^2 processors; Pavel and Akl [14] presented an algorithm running in $O(logN)$ time using N^2 processors. Chen [6] presented a work-optimal $O(NlogN)$ time algorithm using $\frac{N}{logN}$ EREW pro-

cessors. Fujiwara et al. [7] also presented a work-optimal algorithm running in $O(\frac{logN}{loglogN})$ time using $\frac{N^2 loglogN}{logN}$ CRCW processors and an algorithm in $O(logN)$ time using $\frac{N^2}{logN}$ EREW processors. Hayashi et al. [8] presented a work-optimal algorithm running in $O(loglogN)$ time using $\frac{N^2}{loglogN}$ CRCW processors. In recent years, several sequential 3-D distance transform algorithms were introduced by [3], [4]. Up to now, very few parallel algorithms are developed for the 3D_EDT computation [12].

The contribution of this paper is to develop work-optimal parallel algorithms for both the 3D_EDT and its applications such as building Voronoi diagram and Voronoi polyhedra, finding all maximal empty spheres and the largest empty sphere, and computing the distance-based MAT for a 3-D binary image of size $N \times N \times N$. All of them can be computed in O(log log N) time using $\frac{N^3}{loglogN}$ CRCW processors and in O(log N) time using $\frac{N^3}{logN}$ EREW processors respectively. To the best of our knowledge, those algorithms are work optimal and are the best results that never found before. In an n-D space, the nD_EDT and its applications as described above for a binary image of size N^n can be computed in $O(nloglogN)$ time using $\frac{N^n}{loglogN}$ CRCW processors or in $O(nlogN)$ time using $\frac{N^n}{logN}$ EREW processors.

The remainder of this paper is organized as follows. In Section 2, we introduce the PRAM models and some notations upon which our algorithms are based. In Section 3, several essential concepts are introduced. In Section 4, we describe our EDT parallel algorithms in details. In Section 5, some applications are introduced. Some concluding remarks are included in the last section.

2 Preliminaries and Notation

2.1 The PRAM Models

The Parallel Random Access Machine (*PRAM*, for short) consists of a number of identical processors, memory access unit, and a shared-memory. The shared-memory stores data and serves as the communication medium for the processors. At each step, every processor performs the same instruction, with a number of processors marked out. The concurrent read concurrent write PRAM (*CRCW*, for short) and exclusive read exclusive write PRAM (*EREW*, for short) models which will be used in this paper for parallel computation are two types of the PRAM models. The Common CRCW PRAM is a submodel of CRCW PRAM. Because a single step execution of the N-processor CRCW can be simulated by an N-processor EREW in $O(logN)$ time, all of the algorithms developed on the CRCW model in this paper will be simulated on the EREW PRAM model with a slowdown of $O(logN)$ times. See [1], [10] for details. Without loss of generality, throughout this paper, the Common CRCW PRAM submodel is used as our CRCW PRAM computational model and we only analyze the complexity of algorithms implemented on the CRCW PRAM model. The complexity analysis for the algorithm to be implemented on the EREW PRAM model can be discussed similarly.

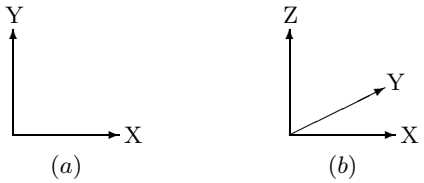


Figure 1: (a). An illustration of 2D axes X and Y. (b). An illustration of 3D axes X, Y and Z.

Lemma 1 *[10] A single step execution of the N-processor CRCW can be simulated by an N-processor EREW in $O(logN)$ time.* □

2.2 Definitions

Figure 1 (a) illustrates the directions of axes in two dimensions X and Y, and Figure 1 (b) depicts the directions of axes in three dimensions X, Y and Z. Throughout this paper, a *pixel* (short for picture element) is used to represent a point in a 2-D plane and a *voxel* (short for volume element) is used to represent a point in a 3-D (or n-D) space. Both of them are represented by their Cartesian coordinates.

Definition 1 *Let $\mathcal{P} = \{p \mid p = (i,j) = 0$ or $1, \ 1 \le i, \ j \le N\}$ denote a 2-D binary image array of size $N \times N$. Let $\mathcal{B}_{2D} = \{q \mid q = (x,y) = 1, \ 1 \le x, \ y \le N\}$ represent the set of the 1-pixels of the 2-D binary image. Then the* two-dimensional Euclidean distance transform (2D_EDT) *of pixel p with respect to its nearest 1-pixel $\mathcal{N}_p$ is denoted by* $\mathrm{edt_p} = \min_{q \in B_{2D}}\{((i-x)^2 + (j-y)^2)^{1/2}\}$ *and the 2D_EDT of $\mathcal{P}$ is to compute $\{edt_p \mid$ for all $p \in \mathcal{P}\}$.* □

Definition 2 *Let* $\mathcal{V} = \{p \mid p = (i,j,k) = 0 \text{ or } 1, \ 1 \le i,j,k \le N\}$ *denote a 3-D binary image array of size* $N \times N \times N$. *Let* $\mathcal{B}_{3D} = \{q \mid q = (x,y,z) = 1, \ 1 \le x,y,z \le N\}$ *represent the set of the 1-voxels of the 3-D binary image. Then the* three-dimensional Euclidean distance transform (3D_EDT) *of voxel p with respect to its nearest 1-voxel* $\mathcal{N}_p$ *is denoted by* $\mathrm{edt_p} = \min_{q \in B_{3D}}\{((i-x)^2+(j-y)^2+(k-z)^2)^{1/2}\}$ *and the 3D_EDT of* $\mathcal{V}$ *is to compute* $\{edt_p \mid \text{for all } p \in \mathcal{V}\}$. □

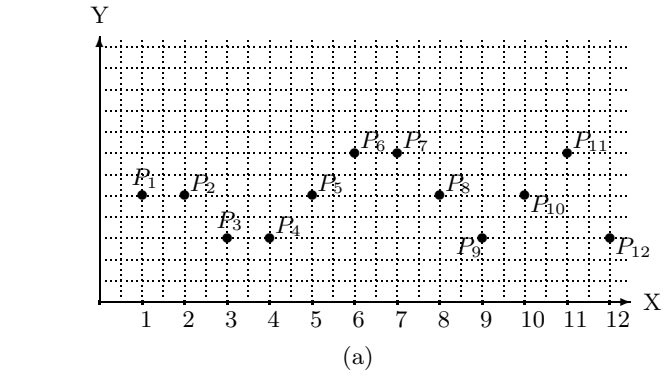
Definition 3 *A voxel in an n-D space can be represented by its coordinate* $(x_1, x_2, ..., x_n)$, *where* $x_1, x_2, ..., x_n$ *represent the value of each axis. Let* $\mathcal{H} = \{p \mid p = (p_1, p_2, ..., p_n) = 0 \text{ or } 1, \ 1 \le p_1, p_2, ..., p_n \le N\}$ *denote an n-D binary image array of size* N^n. *Let* $\mathcal{B}_{nD} = \{q \mid q = (q_1, q_2, ..., q_n) = 1, \ 1 \le q_1, q_2, ..., q_n \le N\}$ *represent the set of the 1-voxels of the n-D binary image. Then the nD_EDT of voxel p with respect to its nearest 1-voxel q is denoted by* $\mathrm{edt_p} = \min_{q \in B_{nD}}\{ (\sum_{i=1}^{n}(p_i - q_i)^2)^{1/2} \}$ *and the nD_EDT is to compute* $\{edt_p \mid \text{for all } p \in \mathcal{H}\}$. □

Definition 4 *Plane* Γ_k *as shown in Figure 3 can be represented as* $\{(x,y,k) \mid 1 \le x,y \le N, \ k \ is \ a \ fixed \ integer\}$ *or simply represented as* $\{(x,y) \mid 1 \le x,y \le N\}$ *if k is well understood. Then,* $\mathcal{V}$ *can be represented by* $\mathcal{V} = \{\Gamma_k \mid 1 \le k \le N\}$. *We can generate a vertical column parallel to Z-axis by picking up a pixel with fixed X- and Y-coordinates* (x,y) *from every* Γ_k *plane,* $1 \le k \le N$. *Because there are* $N \times N$ *pixels on each* Γ_k *plane, so, we can generate total* $N \times N$ *vertical columns, each of them with different combination of x, y values. For convenience, we denote each of these vertical columns as* $Z_{i,j}$-column, Z-column *or* $(x,y,*)$, *and each of these* Γ_k *planes as* Z-plane *or* $(*,*,k)$. □

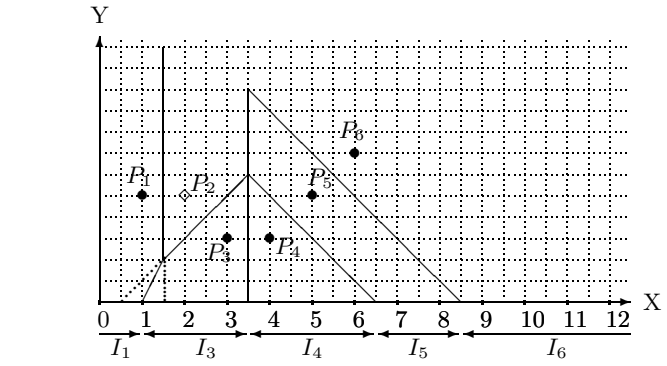
Definition 5 *Let* $\mathcal{H} = \{(x_1, x_2, ..., x_n) \mid 1 \le x_1, x_2, ..., x_n \le N\}$ *be an n-D binary image array of size* N^n. *A specified (n-1)-dimensional binary image array* $\mathcal{U}_k$ *of size* N^{n-1} *can be represented as* $\{(x_1, x_2, ..., x_{n-1}, k) \mid 1 \le x_1, x_2, ..., x_{n-1} \le N, \ where \ k \ is \ a \ fixed \ integer\}$ *or simply represented as* $\{(x_1, x_2, ..., x_{n-1}) \mid 1 \le x_1, x_2, ..., x_{n-1} \le N\}$ *if k is well understood. Then,* $\mathcal{H}$ *can be represented by* $\mathcal{H} = \{\mathcal{U}_k \mid 1 \le k \le N\}$. *We can generate a column (or line) parallel to* X_n*-axis by picking up a voxel with fixed* $x_1, x_2, ..., x_{n-1}$ *coordinate values from every* $\mathcal{U}_k$, $1 \le k \le N$. *Because there are* N^{n-1} *voxels on each* $\mathcal{U}_k$, *so, we can generate total* N^{n-1} *vertical columns, each of them with distinct combination of* $x_1, x_2, ..., x_{n-1}$ *values. For convenience, we denote each of these columns as* $X_{x_1,x_2,...,x_{n-1}}$-column *or* $(x_1, x_2, ..., x_{n-1}, *)$. □

Definition 6 *Let* $\mathcal{N}_R$ *denote the nearest 1-voxel of the voxel* $R = (x_R, y_R, z_R)$ *with respect to all 1-voxels in* $\mathcal{V}$. *Let* $\mathcal{N}_R(\Gamma_k)$, *where* $1 \le k \le N$, *denote the nearest 1-voxel of the voxel R with respect to all 1-voxels in plane* Γ_k. *Let* $(X_{\mathcal{N}_R(\Gamma_k)}, Y_{\mathcal{N}_R(\Gamma_k)}, Z_{\mathcal{N}_R(\Gamma_k)})$ *denote the coordinate of* $\mathcal{N}_R(\Gamma_k)$. *Clearly,* $Z_{\mathcal{N}_R(\Gamma_k)} = k$. □

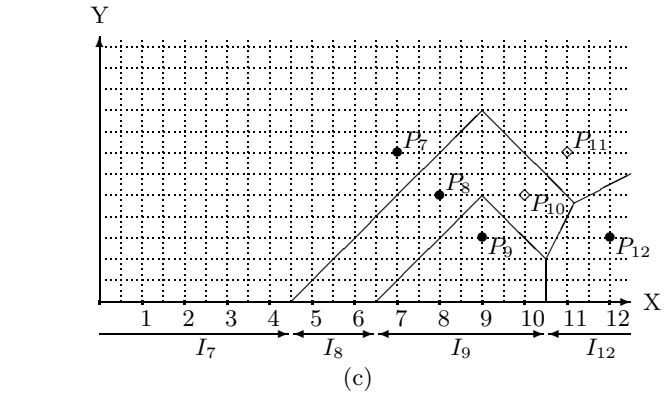
Definition 7 *Given a sequence* $b_1, b_2, \ ..., \ b_N$ *of zeros and ones, to determine the closest 1-pixel to every pixel in this sequence is defined as the* 1-D nearest-one problem. *Let* $\mathcal{P} = \{P_k \mid 1 \le k \le N\}$ *be a set of N pixels in the plane sorted by the coordinate of X-axis. Without loss of generality, we assume that all pixels in* $\mathcal{P}$ *have distinct X-coordinates. A point* P_k, *where* $1 \le k \le N$, *is a* proximate point *of* $\mathcal{P}$ *if there exists a point on the X-axis closer to* P_k *than to any other points in* $\mathcal{P}$. *The* proximate points problem *is to determine all proximate points in* $\mathcal{P}$. *Let's define the proximate points problem of size N with all these N points located in a 2-D space to be the* 2-D proximate points problem. *For example, there are twelve pixels in* $\mathcal{P}$ *as shown in Figure 2(a), and* P_1, P_3, P_4, P_9, *and* P_{12} *are proximate points of* $\mathcal{P}$ *as shown in Figure 2(e). If the proximate points problem of size N with all these N points is located in a 3-D space, then we define this problem to be the* 3-D proximate points problem. *If the proximate points problem of size N with all these N points located in an n-D space, then we define this problem to be the* n-D proximate points problem. *For every point* P_i *of* $\mathcal{P}$, *the locus of all the points in the plane that are closer to* P_i *than to any other points in* $\mathcal{P}$ *is defined to be the* Voronoi cell *of* P_i *and is denoted by* $\mathcal{V}or(i)$. *The collection of all the Voronoi cells of points in* $\mathcal{P}$ *partitions the plane into the* Voronoi diagram *of* $\mathcal{P}$. *The Voronoi diagram of* $\mathcal{P}$ *partitions the X-axis into* proximate intervals. *Let us represent X-axis as* $\{ Q'_k \mid Q'_k = (k,0), \ where \ 1 \le k \le N\}$. *Then the* proximate interval I_i *can be represented as* $\{ Q'_k \mid Q'_k \ is \ closer \ to \ P_i \ than \ to \ any \ other \ points \ in \ \mathcal{P}, \ where \ 1 \le k \le N\}$. *That is, the proximate interval* I_i *is the locus of all the points* Q'_k *if* P_i *is a proximate point. For example, in Figure 2(e), any*



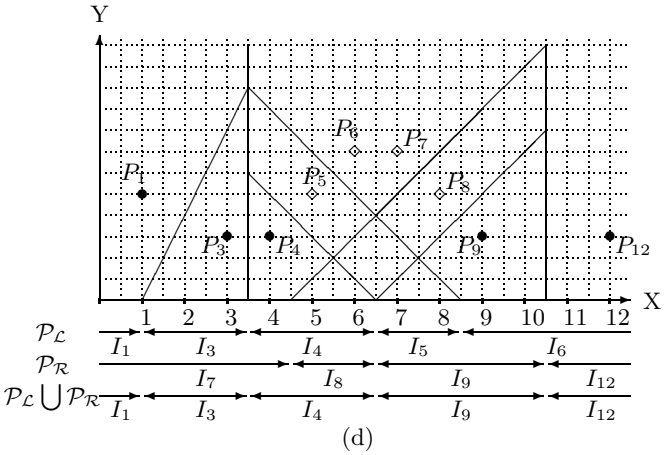
(a)



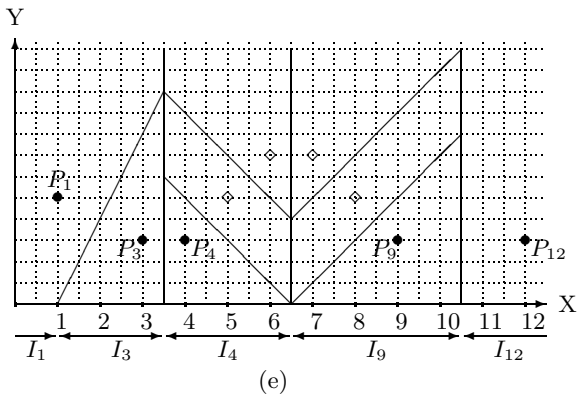
(b)



(c)



(d)



(e)

Figure 2: (a). An illustration of Definition 7 and Theorem 4. (b). An illustration of the set of proximate points $\mathcal{P}_\mathcal{L} = \{P_1, P_3, P_4, P_5, P_6\}$. (c). An illustration of the set of proximate points $\mathcal{P}_\mathcal{R} = \{P_7, P_8, P_9, P_{12}\}$. (d). Illustrating the contact points between two sets of proximate points $\mathcal{P}_\mathcal{L} \bigcup \mathcal{P}_\mathcal{R}$. (e). Illustrating the proximate points of $\mathcal{P}_\mathcal{L} \bigcup \mathcal{P}_\mathcal{R} = \{P_1, P_3, P_4, P_9, P_{12}\}$.

one of intervals $I_1, I_3, I_4, I_9, I_{12}$ is a proximate interval. A point Q'_k is termed as a boundary point *if two proximate intervals I_i and I_j are adjacent and intersected at point Q'_k. Clearly, point Q'_k is equidistant to proximate points P_i and P_j.* □

The following example is a brief description for finding proximate points. Assume that there are a set of twelve pixels as shown in Figure 2(a). Finding the proximate points out of these twelve pixels can be computed by using divide and conquer algorithm as follows. First, we divide these twelve pixels into two sets as shown in Figure 2(b) and Figure 2(c). Then, divide the set $\{P_1, P_2, P_3, P_4, P_5, P_6\}$ in Figure 2(b) into $\{P_1, P_2, P_3\}$ and $\{P_4, P_5, P_6\}$, and divide the set $\{P_7, P_8, P_9, P_{10}, P_{11}, P_{12}\}$ as shown in Figure 2(c) into $\{P_7, P_8, P_9\}$ and $\{P_{10}, P_{11}, P_{12}\}$. Now let's focus on the set $\{P_1, P_2, P_3\}$ in Figure 2(b). We draw a perpendicular bisector for line segment $\overline{P_1P_2}$ and assume that this perpendicular bisector intersects X-axis at coordinate $(X_{P_1P_2}, 0)$. We also draw a perpendicular bisector for line segment $\overline{P_2P_3}$ and assume that this perpendicular bisector intersects X-axis at coordinate $(X_{P_2P_3}, 0)$. The relations among X-coordinates of P_1, P_2, P_3 are $X_{P_1} < X_{P_2} < X_{P_3}$. Then, if point $(X_{P_2P_3}, 0)$ is at the left-hand side of point $(X_{P_1P_2}, 0)$, that is, $X_{P_2P_3} < X_{P_1P_2}$, we say that pixel P_2 is dominated by its left-hand side neighbor P_1 and right-hand side neighbor P_3. All the sets $\{P_4, P_5, P_6\}$, $\{P_7, P_8, P_9\}$ and $\{P_{10}, P_{11}, P_{12}\}$ will be processed in the same way as described above. Clearly,

P_{11} is dominated by its left-hand side neighbor P_{10} and right-hand side neighbor P_{12}. Any pixel dominated by its both side neighbors will cease to be a proximate point. Next, we merge the sets $\{P_1, P_3\}$ and $\{P_4, P_5, P_6\}$ as shown in Figure 2(b) and apply the same procedure as stated above to get the set of proximate points $\mathcal{P}_{\mathcal{L}} = \{P_1, P_3, P_4, P_5, P_6\}$. Also, we merge the sets $\{P_7, P_8, P_9\}$ and $\{P_{10}, P_{12}\}$ as shown in Figure 2(c) and get the set of proximate points $\mathcal{P}_{\mathcal{R}} = \{P_7, P_8, P_9, P_{12}\}$. Clearly, P_{10} is dominated by its left-hand side neighbor P_9 and right-hand side neighbor P_{12} during this merge procedure. Finally, we merge $\mathcal{P}_{\mathcal{L}} = \{P_1, P_3, P_4, P_5, P_6\}$ and $\mathcal{P}_{\mathcal{R}} = \{P_7, P_8, P_9, P_{12}\}$ as shown in Figure 2(d) and get the set of proximate points $\mathcal{P}_{\mathcal{L}} \bigcup \mathcal{P}_{\mathcal{R}} = \{P_1, P_3, P_4, P_9, P_{12}\}$ as shown in Figure 2(e), with $P_i = P_4$ and $P_j = P_9$. Here P_i and P_j are the *contact points* between $\mathcal{P}_{\mathcal{L}}$ and $\mathcal{P}_{\mathcal{R}}$. Clearly, no pixels in $\mathcal{P}_{\mathcal{L}}$ to the right of P_4 can be the proximate points of $\mathcal{P}_{\mathcal{L}} \bigcup \mathcal{P}_{\mathcal{R}}$. Also, no pixels in $\mathcal{P}_{\mathcal{R}}$ to the left of P_9 can be the proximate points of $\mathcal{P}_{\mathcal{L}} \bigcup \mathcal{P}_{\mathcal{R}}$. Each one of the survived proximate points owns its corresponding proximate interval. For example, in Figure 2(e), any one of intervals $I_1, I_3, I_4, I_9, I_{12}$ is a proximate interval.

3 Essential Concepts

Theorem 1 *[12] Let Q_k, $1 \leq k \leq N$ be a sequence of voxels in the same $Z_{i,j}$-column with each located at plane Γ_k respectively. Let $\mathcal{N}_{Q_k}$ be the nearest 1-voxel of voxel Q_k in the 3-D space. Then $\mathcal{N}_{Q_k} \in \{\mathcal{N}_{Q_r}(\Gamma_r) \mid 1 \leq\ k,\ r \leq N\}$.*

Proof: *See Figure 3 for the illustration of this theorem. The proof is omitted.* □

Theorem 2 *Let $Q_k = (X_{Q_k},\ Y_{Q_k},\ k)$ be a sequence of voxels each located at plane Γ_k, $1 \leq k \leq N$, with all of the voxels in the same $Z_{i,j}$-column. Suppose the nearest 1-voxel $\mathcal{N}_{Q_k}$ of voxel Q_k is located at plane Γ_j in the 3-D space. Instead of finding the nearest 1-voxel of voxel Q_k from the original 3-D space, first map the voxel Q_k and its nearest 1-voxel $\mathcal{N}_{Q_k}$ to the pixel $Q_k{}'$ with coordinate $(k, 0)$ and the pixel $P_j{}'$ with coordinate $(j,\ |\overline{Q_j \mathcal{N}_{Q_j}(\Gamma_j)}|)$ respectively in a 2-D space.*

Proof: *See Figure 3 and Figure 4 for the illustration of this theorem. The proof is omitted in this version.* □

Following Theorem 2, we have the following corollary.

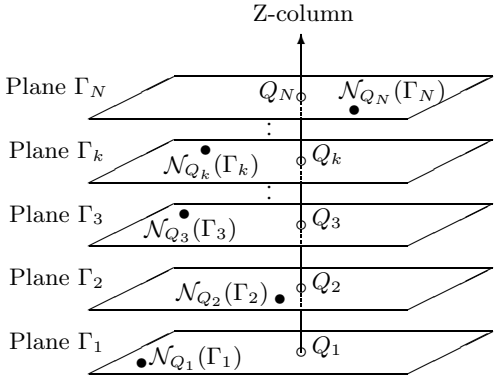


Figure 3: An illustration of Theorem 1 and Theorem 2.

Corollary 1 *After the nearest 1-voxel of each voxel Q=(i, j, k) is found in each independent plane Γ_k, the problem of finding the nearest 1-voxel of the voxel located in each $Z_{i,j}$-column can be reduced to the 2-D proximate points problem of size N.* □

The above theorem and corollary can be extended to an n-D space as follows.

Theorem 3 *Let $Q_k = (x_{1Q_k}, x_{2Q_k}, ..., x_{n-1Q_k}, k)$ be a sequence of voxels each located at (n-1)-D space $\mathcal{U}_k$, $1 \leq k \leq N$, with all of the voxels in the same $X_{x_1,x_2,...,x_{n-1}}$-column. Suppose the nearest 1-voxel $\mathcal{N}_{Q_k}$ of voxel Q_k is located at the (n-1)-D space $\mathcal{U}_j$ with an n-D coordinate $\mathcal{N}_{Q_k} = (x_{1\mathcal{N}_{Q_k}}, x_{2\mathcal{N}_{Q_k}}, ..., x_{n-1\mathcal{N}_{Q_k}}, j)$. Instead of finding the nearest 1-voxel of voxel Q_k from the original n-D space, first map the voxel Q_k and its nearest 1-voxel $\mathcal{N}_{Q_k}$ to the pixel $Q_k{}'$ with coordinate $(k, 0)$ and the pixel $P_j{}'$ with coordinate $(j,\ |\overline{Q_j \mathcal{N}_{Q_j}(\mathcal{U}_j)}|)$ respectively of a 2-D space. Then based on the mapping, the nearest 1-voxel of voxel Q_k located in the n-D space can be found as the pixel $P_j{}'$ located in the newly created 2-D space.*

Proof: *See Theorem 2.* □

Corollary 2 *After the nearest 1-voxel $\mathcal{N}_{Q_k}(\mathcal{U}_k)$ of each voxel $Q_k = (x_1, x_2, ..., x_{n-1}, k)$ is found in each independent (n-1)-dimensional space $\mathcal{U}_k$, the problem of finding the nearest 1-voxel $\mathcal{N}_{Q_k}$ of the voxel Q_k located in each $X_{x_1,x_2,...,x_{n-1}}$-column can be reduced to the 2-D proximate points problem of size N.* □

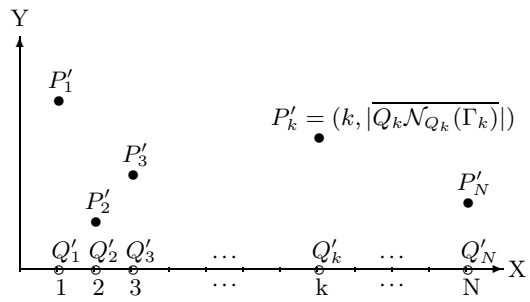


Figure 4: An illustration of Theorem 2.

4 Optimal Parallel Algorithms for 3D_EDT

The idea of our algorithm is based on the *dimensionality reduction* technique. That is, instead of computing the 3D_EDT from a 3-D space directly, we first compute the 1D_EDT of all $N \times N \times N$ columns, then compute the 2D_EDT of all $N \times N$ Z-planes based on the result get from 1D_EDT, finally compute the 3D_EDT based on the messages get from the 1D_EDT and 2D_EDT computations. The following is the detailed description of our algorithms.

4.1 Description of Algorithm

First, based on Lemma 2 as derived by *JáJá* and *Vishkin*, we solve the 1-D nearest-one problem as stated in Lemma 3.

Lemma 2 *[10], [17] The task of computing the prefix maxima (prefix minima) of an N-item sequence can be performed in $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N}{logN}$ EREW processors.* □

Lemma 3 *The 1-D nearest-one problem (i.e., 1D_EDT) of size N can be solved in $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N}{logN}$ EREW processors.* □

Let d_j denote the distance between a pixel $Q_i = (i,\ j)$ and its 1-D nearest-one pixel $\mathcal{N}_{Q_i} = (i,\ \mathcal{N}_j)$ located at the same plane Γ with the same column i, then $d_j = |\overline{j\mathcal{N}_j}|$. If no 1-pixel is in $\{(i,1),(i,2),...,(i,N)\}$, let $\mathcal{N}_j = +\infty$. There are N columns for a 2-D $N \times N$ binary image array. After we apply Lemma 3 for all the N columns in parallel, every pixel has computed its d_j.

Next, we describe the 2D_EDT computation in details as follows. For plane Γ, by mapping the pixel Q_i located at $(i,\ j)$ and its corresponding 1-D nearest-one pixel $\mathcal{N}_{Q_i}$ located at $(i,\ \mathcal{N}_j)$ to the pixels Q'_i located at $(i,\ 0)$ and P'_i located at $(i,\ d_j)$ of the newly created coordinate system respectively, the 2D_EDT problem is reduced to the proximate points problem with $\mathcal{P}_j = \{P'_i = (i, d_j) \mid 1 \leq i \leq N\}$ as defined in Definition 7. The proximate points problem can be solved by Theorem 4 [8]. Corollary 3 is a direct result from Theorems 3, 4, Corollaries 1, 2 and Theorem 4.

Theorem 4 *[8] The 2-D proximate points problem of size N can be solved in $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N}{logN}$ EREW processors.* □

Corollary 3 *For each $Z_{i,j}$-column, $1 \leq i,\ j \leq N$, of the 3-D proximate points problem of size N in the 3-D space, it can be solved in $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N}{logN}$ EREW processors. For each $X_{x_1,x_2,...,x_{n-1}}$-column, $1 \leq x_1, x_2, ..., x_{n-1} \leq N$, of the* n-D *proximate points problem of size N in the n-D space, it can be solved in $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N}{logN}$ EREW processors also.* □

Having solved the 2-D proximate points problem in parallel, we can determine how many pixels of $\mathcal{P}_j$ stated above are proximate points and to which proximate interval I_i the pixel Q'_i is belonged. Now we have gotten the coordinate of the 2-D nearest 1-pixel for every pixel of row j on the 2-D image plane. The parallel algorithm for computing 2D_EDT is stated as follows.

ALGORITHM *2D_EDT_CRCW*

Input: A 2-D $N \times N$ binary image array, each pixel being represented by $p = (i, j) = 0$ *or* 1, for $1 \leq i,\ j \leq N$.

Output: A 2-D $N \times N$ array, each pixel $p = (i, j)$ being represented by $\mathcal{N}_p = (X_{\mathcal{N}_p}, Y_{\mathcal{N}_p})$, for $1 \leq i,\ j \leq N$, or edt_p.

1.1 for $i := 1$ **to** N **pardo**

1.2 Apply Lemma 3 to solve the 1-D nearest-one problem for every column i of a 2-D $N \times N$ binary image array.

1.3 end;

2.1 for $j := 1$ **to** N **pardo**

2.2 Construct the set $\mathcal{P}_j$ of pixels for every row j, where $\mathcal{P}_j = \{P'_i = (i, d_j) \mid 1 \leq i \leq N\}$, $1 \leq j \leq N$, for the proximate points problem computation.

2.3 Apply Theorem 4 to solve the proximate points problem, for every row j in parallel.

2.4 end;

Finally, the parallel algorithm for computing 3D_EDT consists of two phases, i.e., the plane phase and the vertical phase. During the plane phase, for each Z-plane Γ_k, $1 \leq k \leq N$, we find the nearest 1-pixel for each pixel in the plane. This is a 2D_EDT problem which can be implemented by ALGORITHM *2D_EDT_CRCW*. Let $Q_k = (i,\ j,\ k)$, $1 \leq i,\ j,\ k \leq N$, be a voxel with coordinate $(i,\ j,\ k)$ located at plane Γ_k, and $\mathcal{N}_{Q_k}(\Gamma_k) = (x,\ y,\ k) \in \Gamma_k$ be the nearest 1-voxel of the voxel Q_k at plane Γ_k as shown in Figure 3. Then the distance from Q_k to its nearest 1-voxel $\mathcal{N}_{Q_k}(\Gamma_k)$ in the same Γ_k plane is $((i-x)^2 + (j-y)^2)^{1/2}$. In the vertical phase, we first generate $Z_{i,j}$-column. Because there are $N \times N$ pixels on each Γ_k plane, we can construct total $N \times N$ vertical columns, each of them with different combination of x, y values. we then use the $Z_{i,j}$-column to find out where the nearest 1-voxel of each voxel is located. In Theorem 1, we have proved that for every voxel $Q_k = (X_{Q_k},\ Y_{Q_k},\ k)$, the nearest 1-voxel of voxel Q_k must be one of $\mathcal{N}_{Q_1}(\Gamma_1)$, $\mathcal{N}_{Q_2}(\Gamma_2)$, ..., $\mathcal{N}_{Q_N}(\Gamma_N)$. In Theorem 2, we have shown that for computing the 3-D nearest 1-voxel of voxel Q_k, the voxel Q_k and voxel $\mathcal{N}_{Q_k}(\Gamma_j)$ in the 3-D space as shown in Figure 3 can be mapped into the pixel Q'_k on X-axis with coordinate $(k, 0)$ and the pixel P'_j with coordinate $(j, |\overline{Q_j \mathcal{N}_{Q_j}(\Gamma_j)}|)$ in the 2-D plane as shown in Figure 4. Based on Corollary 1, the 3-D nearest 1-voxel problem for a $Z_{i,j}$-column is reduced to the proximate points problem. All $N \times N$ $Z_{i,j}$-columns will be processed in parallel. Then, apply Theorem 4 to solve the proximate points problem for all $N \times N$ $Z_{i,j}$-columns in parallel. After this step, we have gotten the coordinate of the nearest 1-voxel of every voxel on the 3-D space. The parallel algorithm for computing 3D_EDT is stated as follows.

ALGORITHM *3D_EDT_CRCW*

Input: A 3-D $N \times N \times N$ binary image array, each voxel being represented by $Q = (i, j, k) = 0$ *or* 1, for $1 \leq i, j, k \leq N$.

Output: A 3-D $N \times N \times N$ array, each voxel Q=(i, j, k) being represented by $\mathcal{N}_Q$=($X_{\mathcal{N}_Q}$,$Y_{\mathcal{N}_Q}$,$Z_{\mathcal{N}_Q}$) or edt_Q.

1.1 for $k := 1$ **to** N **pardo**

1.2 Apply ALGORITHM *2D_EDT_CRCW* on each Z-plane Γ_k, $1 \leq k \leq N$.

1.3 end;

2.1 for $i,\ j := 1$ **to** N **pardo**

2.2 Construct all $Z_{i,j}$-columns, $1 \leq i,\ j \leq N$ in parallel.

2.3 Map each voxel on $Z_{i,j}$-column together with its nearest 1-voxel $\mathcal{N}_{Q_k}(\Gamma_k)$ from the original coordinate system to the newly created 2-D coordinate system as described above.

2.4 Apply Theorem 4 to solve the proximate points problem for all $N \times N$ $Z_{i,j}$-columns in parallel.

2.5 end;

4.2 Time Complexity Analysis

The complexity analysis of ALGORITHM *2D_EDT_CRCW* is stated as follows. Step 1 can be performed in $O(loglogN)$ time using $\frac{N^2}{loglogN}$ CRCW processors. Both step 2.2 and step 2.3 can be performed in $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors. Therefore, processing all N rows in parallel for step 2 will take $O(loglogN)$ time using $\frac{N^2}{loglogN}$ CRCW processors. Then, we have the following result.

Theorem 5 *[8] The EDT of a binary image of size $N \times N$ can be computed in $O(loglogN)$ time using $\frac{N^2}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N^2}{logN}$ EREW processors.* □

According to Definition 7, if we assign every pixel $p = (i, j)$ the coordinate $(X_{\mathcal{N}_p}, Y_{\mathcal{N}_p})$ instead of edt_p, then the output of ALGORITHM *2D_ EDT_CRCW* turns out to be the set of $\{ \mathcal{V}or(p) \mid p = (i, j), 1 \leq i, j \leq N\}$. This concludes the following corollary.

Corollary 4 *The Voronoi polygons and Voronoi diagram of a binary image of size $N \times N$ can be constructed in $O(loglogN)$ time using $\frac{N^2}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N^2}{logN}$ EREW processors.* □

We then analyze the complexity of ALGORITHM $3D\_EDT\_CRCW$. In the plane phase, we apply ALGORITHM $2D\_EDT\_CRCW$ on each Z-plane Γ_k, $1 \leq k \leq N$, in parallel. According to Theorem 5, it can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors. Based on Theorem 2, step 2.3 will take $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors for processing $N \times N$ $Z_{i,j}$-columns in parallel. Step 2.4 applies Theorem 4 to solve the proximate points problem for all $N \times N$ $Z_{i,j}$-columns in parallel and it can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors. This concludes the following theorem and corollary.

Theorem 6 *The EDT of a binary image of size $N \times N \times N$ can be computed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N^3}{logN}$ EREW processors.* □

Corollary 5 *The Voronoi polyhedra and Voronoi diagram of a binary image of size $N \times N \times N$ can be constructed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N^3}{logN}$ EREW processors.* □

Like the 3D_EDT problem, the nD_EDT computation can be computed by applying the dimensionality reduction technique. First, according to Lemma 3, it takes $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors for the 1D_EDT computation. Suppose it takes $O((i-1)loglogN)$ time using $\frac{N^{(i-1)}}{loglogN}$ CRCW processors for the $(i-1)D\_EDT$ computation. It is easy to prove that it takes $O(iloglogN)$ time using $\frac{N^i}{loglogN}$ CRCW processors for the iD_EDT computation. We skip the detailed analysis. This leads to the following theorem.

Theorem 7 *The nD_EDT of a binary image of size N^n can be computed in $O(nloglogN)$ time using $\frac{N^n}{loglogN}$ CRCW processors or in $O(nlogN)$ time using $\frac{N^n}{logN}$ EREW processors.* □

5 Applications

Our parallel algorithms for EDT computation can be used in all the application areas where the EDT is used as their elementary operations.

5.1 All Maximal Empty Spheres and The Largest Empty Sphere

An empty sphere is a sphere whose interior contains only 0-voxel. A maximal empty sphere is an empty sphere that contains in no other empty sphere. The *all maximal empty spheres* problem is to find all the maximal empty spheres in a 3-D binary image. The *largest empty sphere* problem is to find the largest empty sphere from all the maximal empty spheres.

ALGORITHM *All_Maximal_Empty_Spheres*

Input: A 3-D $N \times N \times N$ binary image array $\mathcal{V}$, each voxel being represented by $Q = (i, j, k)$ = 0 or 1, for $1 \leq i,\ j,\ k \leq N$.

Output: Each voxel $Q = (i, j, k)$ is represented by its labelled r_Q to represent that there exists a maximal empty sphere with radius r_Q and origin Q. The largest empty sphere of all the maximal empty spheres of a binary image of size $N \times N \times N$ is represented by radius max_r_Q and origin Q.

1.1 **for** $i, j, k := 1$ **to** N **pardo**

1.2 Apply ALGORITHM $3D\_EDT\_CRCW$ to compute the 3D_EDT of $\mathcal{V}$.

1.3 **end;**

2.1 **for** $i,\ j,\ k := 1$ **to** N **pardo**

2.2 Assume the largest radius of an empty sphere centered at every voxel $Q = (i, j, k)$ is r_Q. Then r_Q= min {*i-1, j-1, k-1, N-i, N-j, N-k, edt_Q*} for every voxel Q.

2.3 **end;**

3.1 **for** $i,\ j,\ k := 1$ **to** N **pardo**

3.2 Check whether there exists a neighboring voxel $Q' = (i', j', k')$ such that the sphere with radius r_Q and origin Q is covered by the sphere with radius $r_{Q'}$ and origin Q'. If no such sphere exists, label the sphere with radius r_Q and origin $Q = (i, j, k)$ as a maximal empty sphere. (Note: A voxel Q with coordinate (i, j, k) has 26 neighboring voxels with coordinate (i', j', k'), where $|i - i'| \leq 1$ *and* $|j - j'| \leq 1$ *and* $|k - k'| \leq 1$.)

3.3 **end;**

4.1 **for** $i,\ j,\ k := 1$ **to** N **pardo**

4.2 Compute $max\_r_Q = \max \{ r_Q \mid$ where r_Q is the radius of the labelled voxel Q computed in step 3$\}$. Then the voxel $Q = (i, j, k)$ with the largest radius max_r_Q is the largest empty sphere in $\mathcal{V}$.

4.3 end;

Step 1 can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors. For both step 2 and step 3, we first partition the N^3 voxels of $\mathcal{V}$ into $loglogN$ subsets, each subset containing $\frac{N^3}{loglogN}$ voxels. Then, assign $\frac{N^3}{loglogN}$ CRCW processors to a subset with each processor assigned to a voxel (i, j, k) to compute r_Q by finding the minimum value from the set $\{i-1, j-1, k-1, N-i, N-j, N-k, edt_Q\}$ in parallel for step 2 and to check if radius r_Q is the maximum among its 26-neighbors in parallel for step 3. For each processor, we use the optimal sequential algorithm for finding the minimum (or maximum) described in [10]. In the worst case, for each processor it takes $O(loglog7) = O(1)$ time to compute the r_Q and $O(loglog26) = O(1)$ time to check if the r_Q is the maximum. Sequentially applying the procedures described above for all $loglogN$ subsets takes $O(loglogN)$ time. Clearly, both steps 2 and 3 can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors. In step 4, we will find the maximum of all the labelled r_Q computed in step 3. In the worst case, the number of the labelled r_Q could be $O(N^3)$. It can be solved in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors by applying Lemma 4 described in [10]. This concludes the following theorem.

Lemma 4 *[10] Finding the maximum of N elements can be done optimally in $O(loglogN)$ time using $\frac{N}{loglogN}$ CRCW processors.* □

Theorem 8 *Finding all the maximal empty spheres and the largest empty sphere of all the maximal empty spheres of a binary image of size $N \times N \times N$ can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N^3}{logN}$ EREW processors.* □

5.2 The Distance-Based Medial Axis Transform

The distance-based MAT is defined as a recovering of the object by maximal digital disks in a 2-D plane or maximal digital spheres in a 3-D space included in the object. A maximal digital disk (or sphere) is a digital disk (or sphere) that contains in no other digital disk (or sphere). Figure 5 illustrates the distance-based MAT in a 2-D plane. Here we will focus on the parallel computation of the distance-based MAT in a 3-D space. If every voxel of the medial axis of an object is labelled with the radius of its corresponding maximal sphere, then the object can be exactly rebuilt using the information of its axis.

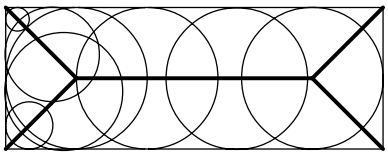


Figure 5: An illustration of the distance-based medial axis transform in a 2-D plane.

ALGORITHM *DB_MAT_CRCW*

Input: A 3-D $N \times N \times N$ binary image array $\mathcal{V}$, each voxel being represented by Q=(i, j, k) =0 *or* 1, for $1 \le i, j, k \le N$.

Output: Each voxel Q=(i, j, k) of the medial axis of an object is labelled with the radius r_Q of its corresponding maximal sphere.

1.1 for $i, j, k := 1$ **to** N **pardo**

1.2 Complement the value of every voxel $Q = (i, j, k)$ of $\mathcal{V}$. That is, a 1-voxel Q is changed to 0-voxel and vice versa. We denote the complemented 3-D $N \times N \times N$ binary image array as $\mathcal{V}'$.

1.3 end;

2.1 for $i,\ j,\ k := 1$ **to** N **pardo**

2.2 Compute all maximal empty spheres for every voxel in the 3-D $N \times N \times N$ binary image array $\mathcal{V}'$ by applying ALGORITHM All_Maximal_Empty_Spheres. After this step, each voxel (i, j, k) of $\mathcal{V}'$ is assigned a maximal empty sphere with radius r_Q and origin (i, j, k). The radius r_Q of the maximal empty sphere of each voxel (i, j, k) of $\mathcal{V}'$ is exactly the radius of the maximal sphere of the voxel on the medial axis of an object in $\mathcal{V}$.

2.3 end;

Clearly, both step 1 and step 2 can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors. This concludes the following theorem.

Theorem 9 *The task of computing the distance-based MAT of a binary image of size $N \times N \times N$ can be performed in $O(loglogN)$ time using $\frac{N^3}{loglogN}$ CRCW processors or in $O(logN)$ time using $\frac{N^3}{logN}$ EREW processors.* □

6 Concluding Remarks

Both the 3D_EDT and its applications such as building Voronoi diagram and Voronoi polyhedra, finding all maximal empty spheres and the largest empty sphere, and computing the distance-based MAT in a 3-D binary image for an image of size $N \times N \times N$ can be computed in O(log log N) time using $\frac{N^3}{loglogN}$ CRCW processors and in O(log N) time using $\frac{N^3}{logN}$ EREW processors respectively. To the best of our knowledge, those algorithms are work optimal and are the best results that never found before. In an n-D space, the nD_EDT and its applications as described above for a binary image of size N^n can be computed in $O(nloglogN)$ time using $\frac{N^n}{loglogN}$ CRCW processors or in $O(nlogN)$ time using $\frac{N^n}{logN}$ EREW processors.

References

[1] S. G. Akl, *Parallel Computation: Models and Methods*, Prentice-Hall, Upper Saddle River, NJ, (1997).

[2] H. Blum, A transformation for extracting new descriptors of shape, In W. Wathen-Dunn, editor, *Models for the Perception of Speech and Visual Form*, MIT Press, Cambridge, Mass., 362-380 (1967).

[3] G. Borgefors, Distance transformations in arbitrary dimensions, *Computer Vision, Graphics, and Image Processing*, vol. 27, 321-345 (1984).

[4] G. Borgefors, On digital distance transforms in three dimensions, *Computer Vision, Graphics, and Image Processing*, vol. 64, 368-376 (1996).

[5] H. Breu, J. Gil, D. Kirkpatrick, and M. Werman, Linear Time Euclidean Distance Transform Algorithms, *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 17, 529-533 (1995).

[6] L. Chen, Optimal algorithm for Complete Euclidean distance Transform, *Chinese J. Computers*, vol. 18, no. 8, 611-616 (1995).

[7] A. Fujiwara, T. Masuzawa and H. Fujiwara, "An optimal parallel algorithm for the Euclidean distance maps of 2-D binary images," *Information Processing Letters*, vol. 54, 295-300 (1995).

[8] T. Hayashi, K. Nakano, and S. Olariu, Optimal Parallel Algorithms for Finding Proximate Points, with Applications, *IEEE Transactions on Parallel and Distributed Systems*, vol. 9, no. 3, 1153-1166 (1998).

[9] T. Hirata and T. Kato, A unified linear-time algorithm for computing distant maps, *Information Processing Letters*, vol. 58, 129-133 (1996).

[10] J. JáJá, *An Introduction to Parallel Algorithms. Addison-Wesley* (1992).

[11] M. N. Kolountzakis, K. N. Kutulakos, Fast computation of the Euclidean distance maps for binary images, *Information Processing Letters*, vol. 43, 181-184 (1992).

[12] Y. H. Lee, S. J. Horng and J. Seitzer, Fast Computation of the *3-D* Euclidean Distance Transform on the EREW PRAM Model, *Proc. 2001 International Conference on Parallel Processing*,(2001).

[13] Y. H. Lee, S. J. Horng, T. W. Kao, F. S. Jaung, Y. J. Chen and H. R. Tsai, Parallel computation of exact Euclidean distance transform, *Parallel Computing*, vol. 22, 311-325 (1996).

[14] S. Pavel and S. G. Akl, Efficient algorithms for Euclidean distance Transform, *Parallel Processing Letters*, vol. 5, no. 2, 205-212 (1995).

[15] A. Rosenfeld and J. L. Pfalz, Sequential operations in digital picture processing, *Journal of the ACM*, vol. 13, 471-494 (1966).

[16] T. Saito and J. I. Toriwaki, New algorithms for Euclidean Distance Transformation of an n-dimensional digitized picture with applications, *Pattern Recognition*, vol. 27, no. 11, 1551-1565 (1994).

[17] Y. Shiloach and U. Vishkin, Finding the Maximum, Merging and Sorting in a Parallel Computation Model, *J. Algorithms*, vol. 2, no. 1, 88-102 (1981).